

30 — Client-Side CSV Export

Requirement: R15, R16, R17 (search results export) **Phase:** N

What it is

When users search for categories or companies in the Admin UI, they sometimes need to take the results out of the browser — to share with a colleague, import into a spreadsheet, or keep as a record. This feature generates a **CSV (Comma-Separated Values) file** entirely in the browser using JavaScript, with no server request involved.

Here are the core concepts:

CSV format

CSV is the simplest tabular data format: each row is a line of text, and columns within a row are separated by commas. The first row is typically the header:

```
ID,Language,RRF Score,Narrative
42,hu,0.0312,"Ipari szivattyuk gyartasa"
42,en,0.0298,"Industrial pump manufacturing"
```

The double-quote wrapping around text fields is important: if the text itself contains commas or newlines, the quotes prevent them from being interpreted as column separators. Double quotes inside a quoted field are escaped by doubling them (`" "` becomes a literal `"`).

Blob API

A **Blob** (Binary Large Object) is a JavaScript object that represents raw binary data. When you create a Blob from a string, the browser allocates memory to hold that data as if it were a file. The Blob itself is not a file on disk — it exists only in memory — but you can create a temporary URL pointing to it.

Object URLs

`URL.createObjectURL(blob)` generates a special `blob:` URL that points to the in-memory Blob. This URL works just like a regular file URL — you can set it as a link's `href`, and clicking the link downloads the Blob's contents as a file. When you are done, `URL.revokeObjectURL(url)` frees the memory.

Dynamic link technique

The standard pattern for triggering a download from JavaScript: 1. Create a Blob with your data 2. Generate an Object URL for it 3. Create an invisible `<a>` element pointing to that URL 4. Set the `download` attribute to your desired filename 5. Programmatically click the link 6. Clean up the URL to free memory

This technique works in all modern browsers and does not require any server involvement.

Real-life analogy

Imagine you are at a restaurant looking at a digital menu on a tablet.

Server-side export would be like asking the waiter to go to the kitchen, print out the menu on paper, and bring it to you. It works, but involves the kitchen (server), takes time, and consumes kitchen resources.

Client-side export is like taking a screenshot of the menu on the tablet and printing it on the restaurant's table printer. All the data is already on your device — you are just reformatting what you already have. The kitchen never gets involved. It is instant, costs nothing on the server side, and works even if the kitchen is busy.

The Blob API is like the tablet's print buffer — it holds the formatted data in memory until the printer (download) is ready. The Object URL is the print queue reference — it gives the printer something to point to. And the programmatic click on the invisible link is like pressing the "Print" button automatically.

Why we used it here

Client-side CSV export is the right choice here for several reasons:

1. **The data is already in the browser.** The search results are loaded and rendered in the React component's state. Generating the CSV from this in-memory data takes milliseconds and zero network traffic.
2. **No server endpoint needed.** Adding a `/search/export` endpoint to the FastAPI backend would require duplicating the search logic (or calling the existing search endpoint internally), handling file streaming, and dealing with memory management for large exports. The client already has the results.
3. **Works offline.** Once the search results are loaded, the export works even if the network connection drops. This is useful for demos or unreliable connections.
4. **Simplicity.** The entire export is 15 lines of JavaScript. A server-side equivalent would involve a new route, a CSV library, response streaming, and content-type headers.

The trade-off: client-side export only works with data already loaded in the browser. If you needed to export 10,000 results but only 10 are displayed, you would need a server-side approach. For this admin panel, the displayed search results (typically 10-20 per page) are the natural unit of export.

Code walkthrough

The complete `exportCsv` function (`admin-ui/src/pages/SearchTest.jsx`)

```
function exportCsv() {
  // Guard: do nothing if there are no results to export
  if (!response?.results?.length) return;

  // Determine the result type – categories and companies have different fields
  const isCategory = endpoint === "category";

  // Step 1: Define the CSV header row based on the result type.
  // Categories have: ID, Language, RRF Score, AI Type, Status, Narrative
  // Companies have: ID, Language, RRF Score, Chunk Index, Partner, Content
```

```

const headers = isCategory
  ? ["ID", "Language", "RRF Score", "AI Type", "Status", "Narrative"]
  : ["ID", "Language", "RRF Score", "Chunk Index", "Partner", "Content"];

// Step 2: Transform each result object into an array of column values.
const rows = response.results.map((r) =>
  isCategory
    ? [
        r.category_id,
        r.language,
        r.rrf_score,
        r.metadata?.ai_type || "", // Optional metadata field
        r.metadata?.status || "", // Optional metadata field
        // Text fields need special CSV handling:
        // 1. Wrap in double quotes so commas inside don't break columns
        // 2. Escape any existing double quotes by doubling them (")
        `"${(r.narrative || "").replace(/"/g, '""')}"`,
      ]
    : [
        r.company_id,
        r.language,
        r.rrf_score,
        r.chunk_index,
        r.is_highlighted ? "Yes" : "No", // Boolean to human-readable
        `"${(r.content_chunk || "").replace(/"/g, '""')}"`,
      ]
);

// Step 3: Build the CSV string.
// Join each row's columns with commas, then join all rows with newlines.
// The header row comes first.
const csv = [headers.join(","), ...rows.map((r) => r.join(","))].join("\n");

// Step 4: Create a Blob – an in-memory file-like object.
// The MIME type "text/csv" tells the browser this is CSV data,
// which helps the OS choose the right application to open it
// (Excel, Google Sheets, etc.)
const blob = new Blob([csv], { type: "text/csv" });

// Step 5: Generate a temporary URL pointing to the Blob.
// This URL looks like: blob:http://localhost:3000/abc-123-def
// It is valid only for this page session.
const url = URL.createObjectURL(blob);

// Step 6: Create a temporary <a> element (not added to the DOM).
// The "download" attribute does two things:
// a) Forces a download instead of navigating to the URL
// b) Sets the suggested filename for the download dialog

```

```

const a = document.createElement("a");
a.href = url;
a.download = `search-results-${endpoint}.csv`;

// Step 7: Programmatically click the link to trigger the download.
// The browser treats this exactly like a user clicking a download link.
a.click();

// Step 8: Free the memory allocated for the Blob URL.
// Without this, the blob stays in memory until the page is closed.
// For small CSVs this barely matters, but it is good practice –
// in a loop or with large data, unreleased URLs cause memory leaks.
URL.revokeObjectURL(url);
}

```

How the function is triggered (`admin-ui/src/pages/SearchTest.jsx`)

The export button appears in the search results header, alongside metadata badges:

```

<button className="btn-secondary btn-sm" onClick={exportCsv}>
  Export CSV
</button>

```

It sits next to the language badge, response time, and retrieval mode indicators, giving the user context about what they are exporting.

Data flow

```

React state (response.results)
  |
  v
Map to arrays (one array per row, one element per column)
  |
  v
Join with commas (each array becomes a CSV line)
  |
  v
Join with newlines (all lines become one CSV string)
  |
  v
Blob (in-memory binary object holding the CSV string)
  |
  v
Object URL (temporary browser-internal URL pointing to the Blob)

```

```
|  
v  
Invisible <a> click (triggers the browser's download mechanism)  
|  
v  
File on disk (search-results-category.csv or search-results-companies.csv)
```

How to explain this in an interview

"We implemented client-side CSV export using the Blob API — the search results are already in the React component's state, so we transform each result into an array of column values, join them with commas and newlines into a CSV string, wrap it in a Blob with a `text/csv` MIME type, and trigger a download by creating a temporary Object URL and programmatically clicking an invisible anchor element. This approach requires zero server involvement, works instantly, and the entire implementation is about 15 lines. We chose client-side over server-side export because the data is already loaded in the browser, and we only need to export the currently displayed page of results. Text fields with commas or quotes are properly escaped with CSV double-quote rules."